LILogic Net: Compact Logic Gate Networks with Learnable Connectivity for Efficient Hardware Deployment

Katarzyna Fojcik[1] Renaldas Zioma[2] Jogundas Armaitis[2]
[1]Wroclaw University of Science and Technology [2]Transcendent Logic
katarzyna.fojcik@pwr.edu.pl rz@TranscendentLogic.com ja@TranscendentLogic.com

Abstract

Efficient machine learning deployment requires models that account for hardware constraints. Because binary logic gates are the fundamental primitives of digital hardware, models built directly from logic operations offer a promising path toward highly energy-efficient computation. Recent work has shown that networks of binary logic gates can be trained with gradient-based optimization and that their wiring can be learned. However, existing approaches remain limited in scalability and training efficiency. We address these challenges by treating the network connectome as a differentiable object and introducing a Top-K connectivity mechanism that enforces structured sparsity during training. Our resulting architecture, LILogicNet, substantially improves the efficiency of logic-gate networks. A model with only 8,000 gates trains on MNIST in under five minutes while achieving 98.45% test accuracy, matching the performance of state-of-the-art logic-gate models that require two orders of magnitude more gates. At larger scales, a 256,000-gate model achieves 60.98% test accuracy on CIFAR-10, surpassing prior approaches with comparable gate budgets. Because the final model is fully binarized and composed entirely of logic operations, inference incurs minimal compute overhead and maps naturally to a wide range of digital hardware platforms, enabling efficient deployment across diverse computing systems.

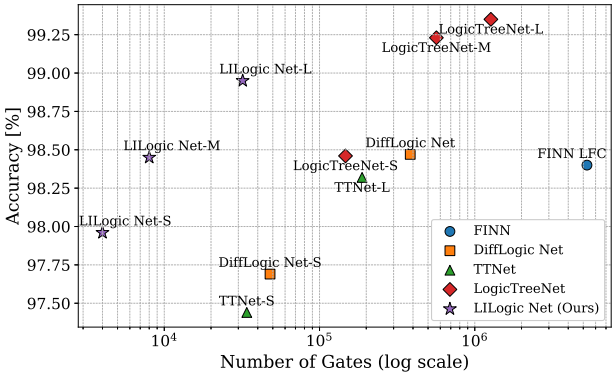


Figure 1. Gate count vs. accuracy on the MNIST dataset. Our models LILogic Nets achieve significantly higher accuracy for a given gate budget compared to state-of-the-art baselines, lying well above the Pareto front.

1. Introduction

In recent years, scaling up machine learning models has driven major advances across real-world applications, enabled by specialized datacenter hardware [14]. In such environments, hardware cost and energy consumption are typically secondary concerns. As machine learning matures, however, there is growing interest in deploying models on low-power, cost-sensitive edge devices [10], demanding tighter co-design between model architecture and hardware capabilities to extend AI beyond traditional datacenter settings [4]. Modern machine learning models largely rely on matrix multiplication operations, efficiently supported by high-level APIs and massively parallel hardware such as GPUs [7, 33]. Computational hardware fundamentally operates on binary logical gate operations, which are directly available across ASICs, FPGAs [18, 23], CPUs, and microcontrollers [5, 22, 30]. Designing models that natively operate on binary signals and logical gates, therefore, represents a promising direction for hardware-efficient AI.

One approach is to replace components of state-of-the-art architectures, such as multilayer perceptrons (MLPs) [9] and convolution layers [19, 46], with logical counterparts. Logical Gate Networks (LGNs), composed of randomly wired layers of trainable two-input logical gates, have shown that purely logical architectures can be trained end-to-end using gradient descent [24]. On benchmarks such as MNIST and CIFAR-10, these models achieved competitive accuracy, though typically with large gate budgets and fixed connectomes. Subsequent work demonstrated that wiring itself can be learned. In particular, Bacellar et al. [3] introduced Differentiable Wiring Networks (DWN), where discrete interconnections between lookup tables are optimized via softmax re-

laxation and discretized at inference. Other studies explored learnable connectomes primarily in tabular settings and from an interpretability perspective [43, 44], but were limited to relatively small networks—up to 2,000 gates—citing training slowdowns due to computational burden.

If LGNs are to serve as practical replacements for MLP components, it is crucial not only to learn connectivity but also to do so efficiently and at scale. In this work, we treat the connectome as a differentiable object and explicitly target scalability and structured sparsity. We introduce a Top-K connectivity mechanism that enforces sparsity during training rather than through post-hoc pruning, improving optimization stability and reducing memory footprint. Furthermore, we propose a Basis Projection (BasisProj) method for efficient gradient-based computation of logic gate outputs.

We introduce **LILogic Net** (*Learnable Interconnect Logic Network*), a compact and scalable LGN architecture achieving 98.95% accuracy on MNIST using only 8,000 logic gates and 60.98% on CIFAR-10 at larger gate budgets. Compared to prior LGN approaches, LILogic Net achieves improved accuracy at comparable or smaller gate counts, demonstrating that (sparse) connectome learning yields measurable end-to-end gains. Code will be made publicly available at https://github.com/jogundas/LILogicNet.

We summarize our contributions as follows:

- **Top-K connectivity**: A mechanism that enforces sparsity during training by selecting the strongest connection among K random candidates per gate.
- **Basis Projection (BasisProj)**: Efficiently computes gate outputs via a fixed projection, yielding up to 4× training speedup.
- **LILogicNet architecture**: Compact logical gate networks that achieve competitive or better accuracy on MNIST, FashionMNIST, and CIFAR-10 with up to two orders of magnitude fewer gates than prior models.

2. Related Work

Several key trends have emerged in binarized and hardware-efficient neural models. Binary Neural Networks (BNNs) [42], such as XNOR-Net [27] and BinaryNet [11], showed that binary weights and activations can significantly reduce memory and computation costs with minimal accuracy loss. In parallel, Quantized Neural Networks [12] reinforced the promise of low-precision optimization. IR-Net [26] addressed information loss in binary training via forward–backward techniques like Libra Parameter Binarization and Error Decay Estimators, while ReActNet [21] improved binary model accuracy through generalized activations. From a deployment perspective, FINN [34] introduced a scalable FPGA framework for real-time BNN inference.

A parallel line of research investigates neural architectures designed explicitly around FPGA lookup tables (LUTs) as the primary computational primitive. Rather than mapping conventional neural layers onto hardware post hoc, these approaches restructure computation to better match the native logic fabric. LUTNet [37] was an early example, using learnable K-input lookup tables to improve logic utilization and enable structured pruning aligned with FPGA resources. LogicNets [35] pushed this further and increased throughput by constraining neuron fan-in so that each unit could be realized directly as a truth table. More recent developments extend this idea beyond binarized weight abstractions. DWNs [3] revisit classical weightless models and introduce learnable address mappings and aggregation mechanisms over symbolic inputs, combining gradient-based training with combinatorial components.

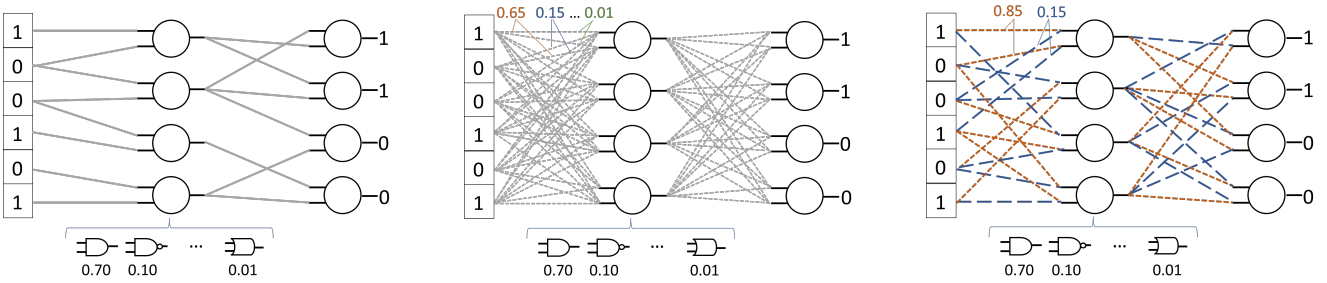
A related direction is differentiable logic-based neural architectures, which replace weighted connections with learnable logic gates. Logic Gate Networks (LGNs), introduced by Petersen et al. [24], showed that such structures can be relaxed and trained with gradient descent. Even early LGNs offered advantages over BNNs in simplicity, interpretability, and hardware alignment. Later work explored their theoretical and practical aspects, including interpretability methods [43, 44], though these models remained limited in scale and performance.

Recent advances have addressed these limitations. Yousefi et al. [41] improved training stability and gate utilization by mitigating the discretization gap using Gumbel noise and Hessian regularization. Petersen et al. [25] proposed TreeLogicNet, incorporating logic gates into convolutional windows with logical OR pooling, achieving 99.35% accuracy on MNIST and 86.29% on CIFAR-10—but using 2–4 orders of magnitude more gates than our method. Although based on logical operations, TreeLogicNet retains CNN-like receptive fields and parameter sharing, resulting in higher gate and operation counts. Other contributions include the eXpLogic framework [39], which enhances interpretability through saliency maps and gate pruning with minimal accuracy drop. Kresse et al. [16] showed that LGNs are inherently suitable for formal verification using SAT solvers, enabling proofs of fairness and robustness, and Clester [6] explored LGNs for generative applications such as music synthesis. On the hardware side, Wang et al. [38] introduced four custom RISC-V instructions to accelerate LGN inference, achieving 95.8% accuracy on MNIST with a compact 18 K logic gate network and reduced runtime by over 87% compared to a standard RV32IM core.

3. Method

3.1. Differentiable Logical Gate Networks

Traditional Logic Gate Networks (LGNs) are sparse, weightless models built from binary logic gates (e.g., AND, XOR). While highly efficient for inference, their discrete nature prevents gradient-based training, thus limiting scalability.



(a) Fixed Connectome (b) Fully Learnable Dense Connectome (c) Top-K Sparse Connectome (here K=2)

Figure 2. Overview of training-time interconnect strategies. Each node learns a distribution over the 16 Boolean functions. Input connections are either fixed or modeled as learnable distributions, depending on the connectome design.

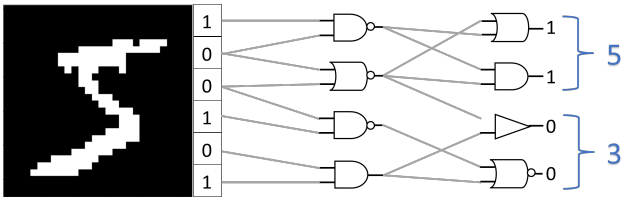


Figure 3. Inference pipeline: all logic gates and connections are fully binarized, yielding a compact, deterministic circuit.

The Differentiable LGN framework introduced by Petersen et al. [24] solves this by: (i) relaxing binary inputs to continuous values in $[0, 1]$, (ii) using probabilistic interpretations of logic gates (e.g., $A \wedge B \approx A \cdot B$ [36, 45]), and (iii) representing each node as a soft combination over all 16 2-input Boolean functions, enabling learning via gradient descent.

This makes LGNs trainable with standard optimizers (e.g., Adam [15]), and—after training—easily discretizable to efficient, purely combinatorial LGNs, which means that they have no "memory" or "state".

We design an LGN architecture that jointly learns the logic gate behavior and the structure of connections between them (the connectome), enabling both efficient training and inference. Our key methodological contributions are twofold: (i) a more GPU-efficient gate representation, and (ii) a systematic comparison of several strategies for interconnect optimization.

3.2. Model Architecture

Our models have 1–4 logic gate layers, each containing a fixed number of gates selected from $\{2\text{K}, 4\text{K}, 8\text{K}, 16\text{K}, 32\text{K}, 64\text{K}, 128\text{K}\}$. Each node receives two relaxed binary inputs and implements one of the 16 Boolean functions [28]. All layers share the same width and interconnect strategy. We consider three strategies for wiring between layers (Figure 2):

- **(F) Fixed connectome:** Each gate receives inputs from a random subset of the previous layer. This mirrors the original Differentiable LGN (DiffLogicNet) [24] and serves as a baseline, typically requiring deeper or wider layers to match accuracy.
- **(L) Fully learnable dense connectome:** For each node, both of its two inputs are connected to all outputs of the previous layer (or to the input vector for the first layer). The connection weights for each input are parameterized separately via a differentiable softmax.
- **(Top-K) Sparse connectome:** For each node, both of its two inputs independently select K random candidate connections from the previous layer (or the input vector for the first layer) at initialization, with each candidate assigned a learnable logit sampled from a standard normal distribution. During training, a softmax over the K candidates produces a weighted combination for each input. We experiment with $K \in \{2, 4, 8, 16, 32, 64, 128\}$ to trade off sparsity and expressiveness.

After training, all models are fully binarized. Each gate selects its most probable logic gate and inputs (for L and Top-K connectomes), yielding a fixed-size, deterministic binary circuit suitable for efficient inference (Figure 3).

3.3. Dataset and Preprocessing

We evaluated our models on the MNIST, FashionMNIST, and CIFAR-10 datasets. The MNIST dataset consists of 60,000 training and 10,000 test images of handwritten digits [20]. Each grayscale image of size 28×28 was flattened into a 784-dimensional input vector and binarized using a fixed intensity threshold of 0.25. The FashionMNIST dataset contains 60,000 training and 10,000 test grayscale images of fashion items across ten categories [40]. Each 28×28 image was flattened and binarized using fixed thresholds $[0.125, , 0.25, , 0.375, , 0.5, , 0.625, , 0.75, , 0.875]$, resulting in an effective input dimensionality of 5,488. The CIFAR-10 dataset contains 50,000 training and 10,000 test color images of 32×32 pixels across ten object categories [17]. For CIFAR-10, each RGB channel was processed independently and binarized using the same thresholds $[0.125, , 0.25, , 0.375, , 0.5, , 0.625, , 0.75, , 0.875]$. The resulting binary maps from all channels were concatenated

into a single flattened vector, yielding an effective input dimensionality of 21,504.

Learning interconnections increases model capacity, which can lead to overfitting in low-data regimes without augmentation. We applied dataset-specific transformations to improve generalization [31]. For MNIST, we used the `torchvision.transforms.v2` interface to apply random rotation ($\pm 10°$), shear (up to $10°$), scaling (by $\pm 10\%$), and elastic deformations with $\alpha = 64$ and $\sigma = 6$. Each augmented transformation was applied independently to generate 10 distinct copies of the training dataset, leading to a $10\times$ expanded training set. For FashionMNIST, no augmentations were applied. For CIFAR-10, we applied standard natural image augmentations [29], including random cropping to 32×32 with 4-pixel reflection padding and random horizontal flips with 50% probability, expanding the training set $8\times$. We note that the original DiffLogicNet baseline [24] does not use augmentations. To enable a fair comparison and isolate the effect of learnable connectivity, we additionally perform evaluations of our models against versions with fixed connectomes under the same augmentation settings.

The validation set was constructed by applying only resizing and binarization (i.e., no augmentation) to the original training images, and the same preprocessing was used for the test set. A fixed random seed ensured reproducibility of the dataset splits and augmentation pipeline.

3.4. Training and Inference

All models were trained for 200 epochs using the Adam optimizer [15] with a learning rate of 0.075, a batch size of 256, and no additional regularization such as dropout or weight decay. Our experiments focus purely on the architectural and interconnect design choices, especially different sparsity regimes, and aim to isolate their effect without the influence of external regularizers. Top-K interconnect variants act implicitly as a form of regularization by constraining the connectivity space.

3.4.1. Differentiable Relaxation

To enable gradient-based training of logic gate selection, each node is parameterized by a vector of 16 real-valued weights, $\mathbf{w} \in \mathbb{R}^{16}$, representing logits over the space of all 16 possible binary logic gates. These logits are transformed into a probability distribution via the softmax function, possibly scaled by a temperature parameter τ_g [1], which we kept fixed at $\tau_g = 1$ in all our experiments:

$$\mathbf{p} = \text{softmax}(\tau_g \cdot \mathbf{w}), \quad p_i = \frac{\exp(\tau_g \cdot w_i)}{\sum_j \exp(\tau_g \cdot w_j)}. \tag{1}$$

The resulting probability vector $\mathbf{p} \in \Delta^{15}$ lies in the 15-dimensional probability simplex, meaning that all entries are non-negative and sum to 1: $\sum_{i=1}^{16} p_i = 1, p_i \geq 0$. This formulation treats gate selection as a categorical distribution that can be trained end-to-end using standard backpropagation. Instead of evaluating each of the 16 gates individually (**FullEval** e.g., $A \cdot B$ for $A \wedge B$), we project this softmax-normalized gate distribution into a lower-dimensional functional basis space (**BasisProj**), enabling more efficient computation and improved interpretability. Specifically, the vector $\mathbf{p}$ is linearly projected via a fixed matrix $W_{16\to4} \in \mathbb{R}^{4\times16}$, yielding:

$$\mathbf{c} = W_{16\to4} \cdot \mathbf{p}, \quad \text{where} \quad \mathbf{c} = [c_1, c_2, c_3, c_4]^\top \tag{2}$$

are the coefficients for the basis $\{1, A, B, A \cdot B\}$.

The gate's continuous output is then computed as:

$$\text{output}_g = c_1 + c_2 A + c_3 B + c_4 (A \cdot B). \tag{3}$$

The fixed projection matrix $W_{16\to4}$ is given by:

$$W_{16\to4}^\top = \left[\begin{array}{r|cccc} \text{Operator} & 1 & A & B & A \cdot B \\ \hline \text{False} & 0 & 0 & 0 & 0 \\ A \wedge B & 0 & 0 & 0 & 1 \\ \neg(A \Rightarrow B) & 0 & 1 & 0 & -1 \\ A & 0 & 1 & 0 & 0 \\ \neg(A \Leftarrow B) & 0 & 0 & 1 & -1 \\ B & 0 & 0 & 1 & 0 \\ A \oplus B & 0 & 1 & 1 & -2 \\ A \vee B & 0 & 1 & 1 & -1 \\ \neg(A \vee B) & 1 & -1 & -1 & 1 \\ \neg(A \oplus B) & 1 & -1 & -1 & 2 \\ \neg B & 1 & 0 & -1 & 0 \\ A \Leftarrow B & 1 & 0 & -1 & 1 \\ \neg A & 1 & -1 & 0 & 0 \\ A \Rightarrow B & 1 & -1 & 0 & 1 \\ \neg(A \wedge B) & 1 & 0 & 0 & -1 \\ \text{True} & 1 & 0 & 0 & 0 \end{array}\right] \tag{4}$$

This formulation offers several advantages. First, it allows efficient parallel computation using dense matrix operations, leveraging GPU acceleration. Implementation does not require platform specific (CUDA) code thus it is more portable. Second, it avoids evaluating 16 separate Boolean functions for each gate, improving training speed and reducing memory usage. Third, the decomposition into basis functions enhances interpretability by exposing the contribution of each term (e.g., A vs. $A \cdot B$). Finally, it provides a flexible foundation for extending the logical expressivity to higher-order gates in future work.

3.4.2. Learnable Interconnections

For the learnable interconnect strategies (Top-K and L), we similarly employ a softmax distribution over input weights to determine input pairings for each gate. For Top-K, a fixed number of candidates K are preselected per gate at initialization, and only the softmax weights over those connections are trained. For the Fully Learnable strategy, the

softmax is applied across all possible inputs, resulting in higher memory and computation costs.

3.4.3. Binarization

After training, we convert the network into a discrete, inference-efficient form by binarizing both gate selection and interconnections. This is done by selecting the maximum probability option (the mode) from the softmax distributions. In practice, this means that each logic gate uses a fixed pair of binary inputs and computes a single Boolean function. Regardless of the training strategy, every node uses exactly two binary inputs at inference time, allowing the model to operate entirely in Boolean logic without any floating-point computation.

3.4.4. Classification Strategy

We adopt a probabilistic majority-voting scheme for classification. The output layer consists of n binary logic gates (e.g., 1,000), evenly divided into k groups corresponding to the k output classes (e.g., 10). During inference, each group counts the number of '1' activations, and the predicted class is the one with the highest count. During training, soft outputs are aggregated per group, and the model is optimized using softmax cross-entropy loss. The sharpness of the soft logic activations is controlled by a global temperature parameter τ, which was empirically tuned for each layer width. The parameter τ affects both convergence and final accuracy and was selected based on the results of a parameter sweep (see Sec. 4.2.4). We use classification accuracy as the evaluation metric, as all tested datasets are balanced and accuracy effectively captures model performance.

4. Experiments

4.1. Full Gate Evaluation vs. Basis Projection

To assess the computational benefits of the proposed projection-based gate evaluation, we compare the **total training time** on the MNIST dataset (for an NVIDIA A4000 GPU) between two strategies:

- **Full Gate Evaluation (FullEval):** Each of the 16 logic gates is evaluated individually per node.
- **Basis Projection (BasisProj):** A single projection into the $\{1, A, B, A \cdot B\}$ basis using a fixed matrix $W_{16\to4}$ replaces per-gate evaluation.

We evaluate six configurations, defined by the number of fixed connectome layers (1F, 2F, 3F) and layer width (8,000 and 32,000 nodes). As shown in Table 1, using basis projection significantly reduces total training time across all tested configurations. The benefit becomes increasingly pronounced with deeper and wider architectures. For instance, in the 32,000 / 3F configuration, the projection-based approach completes training nearly **4× faster** than the full gate evaluation. This speed-up stems from avoiding 16 separate logic gate computations per node in favor of a single dense matrix-vector multiplication—an operation highly optimized for modern accelerators.

Table 1. Full training time (in minutes) and speed-up using basis projection over full gate evaluation. Results are shown for the MNIST dataset and averaged over 10 runs with ≤ 0.1 min variability.

Width	Type	FullEval	BasisProj	Speed-Up
8,000	1F	2.13	1.13	1.89×
	2F	4.75	1.60	2.96×
	3F	7.47	2.21	3.38×
32,000	1F	9.97	2.92	3.41×
	2F	20.78	5.44	3.82×
	3F	31.63	7.99	3.96×

4.2. Interconnect Strategy Analysis

We performed extensive experiments on the MNIST dataset using architectures with 1–4 layers and widths ranging from 2K to 32K, exploring various connectome variants. MNIST's low computational cost allowed us to thoroughly analyze these designs, providing insights that informed the selection of a reduced set of architectures for FashionMNIST and CIFAR-10 experiments. Test accuracy was measured at binarized inference, averaged over at least 5 runs.

4.2.1. Empirical Comparison Across Layer Widths and Depths on MNIST

As shown in Table 2, random and non-trainable F interconnects consistently perform the worst. Their inflexibility requires significantly wider or deeper networks to achieve moderate accuracy, illustrating the limitations of static wiring. Top-K sparse interconnects, on the other hand, deliver strong results across nearly all configurations. Even under extreme sparsity (e.g., Top8), models reach over **98.6%** accuracy when wide or deep enough, outperforming L interconnects in many regimes. The best-performing configuration overall—a Top32 model with two 16K-wide layers (2Top32-16K)—achieves **98.95%** accuracy. Figure 4 illustrates how test accuracy scales with depth for different interconnect strategies and layer widths.

4.2.2. Convergence of Top-K to Dense Connectivity

Figure 5 presents a representative case on the MNIST dataset of a 1-layer model with 2K gates, illustrating how Top-K sparse interconnect performance improves with K, approaching that of the fully learnable (L) connectome. Accuracy rises quickly for small K and saturates as K increases; a logistic fit in $\log_2(K)$ space captures this trend. For $K \geq 128$, performance nearly reaches the fully learnable ceiling, reflecting the general trade-off between sparsity and accuracy.

Table 2. Test accuracy results on MNIST across different layer widths and number of layers for various connectome methods. The results are averaged over at least 5 runs, all with $\leq 0.1\%$ variation. Bolded values indicate the best performance per block.

Layer Width	Layer Type	1	2	3	4
2,000	F	87.09	90.82	93.69	95.38
	Top8	94.29	96.97	97.47	97.59
	Top32	96.38	97.50	**97.83**	**97.71**
	Top128	97.02	**97.66**	97.67	96.92
	L	**97.25**	97.37	96.13	92.00
4,000	F	90.16	93.28	95.77	96.90
	Top8	96.27	97.91	98.13	98.22
	Top32	97.54	98.17	**98.38**	**98.29**
	Top128	97.94	**98.28**	98.30	97.56
	L	**97.96**	98.11	97.02	94.41
8,000	F	91.48	94.83	96.83	97.69
	Top8	97.17	98.43	98.60	98.63
	Top32	98.15	98.63	**98.77**	**98.72**
	Top128	98.39	**98.67**	98.72	97.94
	L	**98.45**	98.62	97.87	96.81
16,000	F	93.16	96.43	97.82	98.34
	Top8	98.15	98.74	98.80	**98.87**
	Top32	98.50	**98.95**	**98.86**	98.73
	Top128	98.55	98.89	98.84	98.23
	L	**98.58**	98.82	-	-
32,000	F	94.74	97.41	98.27	98.52
	Top8	98.41	98.75	98.78	98.76
	Top32	**98.58**	98.83	**98.84**	**98.89**
	Top128	98.51	**98.91**	98.83	97.77
	L	98.52	-	-	-

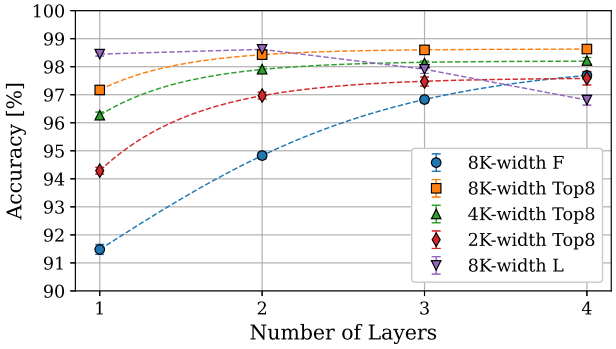


Figure 4. Test accuracy vs. depth for various interconnects.

4.2.3. Sparse Interconnects as Compositional Depth

Sparse interconnect strategies restrict each logic gate to operate on only a small subset of input pairs (e.g., 8 for Top8). While this limits the expressivity of a single layer, stacking layers expands the effective receptive field. In principle, an N-layer Top-K network can access up to K^N input paths—an upper bound that reflects the combinatorial breadth introduced by depth, even though each gate receives only two inputs at inference. Thus, depth enables sparse LGNs to recover some of the flexibility of denser interconnects at much lower cost. For example, two Top8 layers can, in the best case, reach up to 64 input paths, though the learned wiring may cover only part of this space. Depth therefore serves as an effective way to capture richer dependencies while maintaining hardware-efficient sparse connectivity.

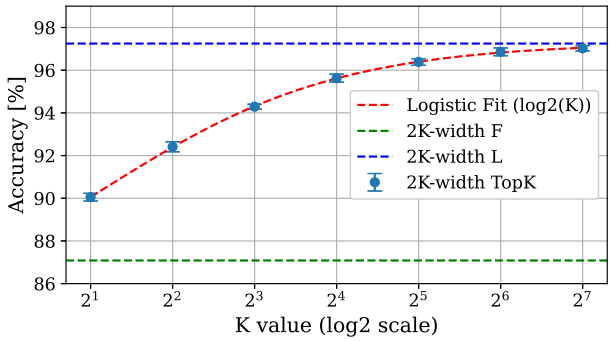


Figure 5. Convergence of 2K-width Top-K test accuracy to L-type connectivity as K increases.

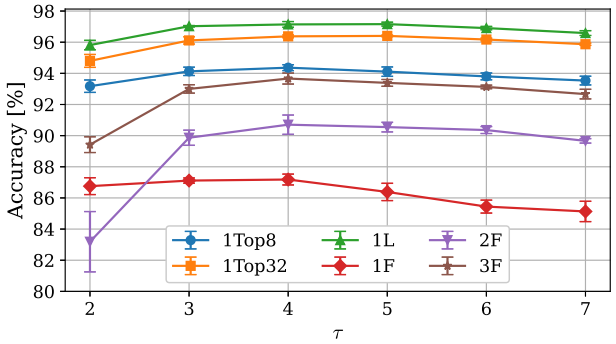


Figure 6. Test accuracy as a function of τ for 2K-width models. Used as a reference for choosing temperature value.

4.2.4. Sensitivity to Temperature Parameters

We observe that the global softmax temperature τ should scale with layer width and dataset complexity (Table 3). For example, based on trends in Figure 6 for 2K-width MNIST architectures, we set $\tau = 4$. It is selected empirically to balance gradient flow during training and confident gate assignments after binarization: lower τ led to overly sharp distributions and vanishing gradients, while higher τ caused unstable training due to excessively flat distributions.

Table 3. LILogicNet experimental results for different datasets and best performing architectures at given gate budgets.

Dataset	Gate Count	Architecture	τ	Memory [KB]	Train Time [min]	Test Accuracy [%]	LUT-6s (LGN)	LUT-6s (LGN +popcount)
MNIST	4K	1L-4K	5	11.7	2.5 ± 0.0	97.96 ± 0.11	1,791	7,103
	8K	1L-8K	10	23.4	4.3 ± 0.1	98.45 ± 0.07	3,491	14,076
	32K	2Top32-16K	10	62.5	57.1 ± 0.2	98.95 ± 0.09	11,833	37,373
FashionMNIST	8K	1L-8K	15	29.3	24.9 ± 0.2	89.95 ± 0.06	3,736	14,321
	64K	2Top32-32K	25	250.0	167.2 ± 2.7	90.26 ± 0.11	22,308	73,938
	128K	2Top32-64K	30	515.6	217.5 ± 3.4	90.61 ± 0.09	39,769	143,226
CIFAR10	8K	1L-8K	20	33.2	8.7 ± 0.1	55.11 ± 0.17	3,830	14,415
	64K	1L-64K	90	266.0	62.9 ± 0.2	57.66 ± 0.17	27.070	104,853
	256K	2Top32-128K	100	1,121.0	107.7 ± 0.1	60.98 ± 0.19	92,353	293,285

Table 4. Comparison of LGN models: FashionMNIST accuracy, FPGA resources, and latency (XC7A200T; XC7Z045 for DWN)

Model	Acc [%]	LUT-6s [$\times 10^3$]	Latency [ns]
DiffLogicNet-S*	89.85	22.7 (10.4)	-
DiffLogicNet*	90.24	172.8 (72.8)	-
DWN (n=6)	89.01	**7.6**	235
LILogicNet-S	89.95	14.3 (3.7)	30
LILogicNet-M	90.26	73.9 (22.3)	43
LILogicNet-L	**90.61**	143.2 (39.8)	-

* Results not reported in the original work; we reproduced the architecture using our framework with the same training setup and training time;
Values in parentheses correspond to the LGN part excluding the popcount;
- Could not fit on FPGA.

4.3. LILogic Net: Accuracy-Efficiency Pareto Front

Based on accuracy, gate count, and training time, we identify three efficient design points for each dataset (Tab. 3). These configurations provide strong predictive performance while maintaining low hardware cost and moderate training time, making them suitable for practical deployment. MNIST and FashionMNIST experiments were conducted on an NVIDIA A4000 GPU, while CIFAR-10 was trained on an NVIDIA H200 GPU to accommodate larger memory requirements for wider architectures. At smaller model scales, increasing layer width yields larger gains than increasing depth, while dense optimization utilizes limited capacity more effectively. However, as the gate budget increases, Top-K models begin to outperform dense alternatives.

Reported training times include the full training procedure with validation but exclude initial data loading and online data augmentation. After enabling `torch.compile`, dense L-type models often train faster than sparse Top-K variants, likely due to better GPU support for dense matrix operations compared to index-heavy sparse computations.

Parameter memory is reported assuming binarized LGNs at inference time with compact bit-packed storage of gate types and input indices. Each node stores its input connections and gate type, resulting in a memory cost proportional to the number of nodes and their fan-in.

We also report the number of LUT6 units required on Xilinx XC7A200T FPGA. We distinguish between the LUT usage for the LGN layers themselves and the total LUT usage including the final population count (popcount) stage. In practice, the popcount adders account for more than two-thirds of the total LUT consumption. To further improve FPGA efficiency, techniques such as the Learnable Reduction method proposed in DWN [3] could be employed to significantly reduce this overhead.

4.3.1. LILogicNet on FPGA

Table 4 compares FPGA resource usage and latency of LGN models on FashionMNIST. For comparable accuracy, LILogicNet requires substantially fewer LUT6 units than DiffLogicNet, though more than DWN due to the cost of the popcount stage. Despite not being optimized specifically for FPGA deployment, LILogicNet achieves low inference latency, demonstrating its practical hardware efficiency.

4.3.2. LILogicNet on ASIC

We taped out 6K-gate LILogicNet-S MNIST variant in SKY130 (SkyWater Technology 130 nm) and SG13 (IHP BiCMOS 130 nm) [49, 50], followed by an 8K-gate LILogicNet-S FashionMNIST variant in GF180 (GlobalFoundries 180 nm) [47, 48]. The designs were implemented using the open-source OpenROAD [2] flow with the respective open PDKs [8, 13, 32].

4.3.3. Multi-Platform Hardware Throughput

Table 5 reports inference throughput on the GPU (NVIDIA A4000), CPU (Xeon Gold 6150, single thread), and FPGA (XC7A200T). The results show that LILogicNet maintains high inference throughput across diverse hardware platforms, reaching hundreds of thousands of FPS on GPU/CPU and tens of millions of FPS on FPGA for compact variants.

Table 5. Inference throughput (t.) measured in frames per second (FPS) for FashionMNIST across different hardware platforms.

Model	GPU t.	CPU t.	FPGA t.
DiffLogicNet-S*	145,472	812	-
DiffLogicNet*	16,063	100	-
LILogicNet-S	826,410	4,827	32,890,000
LILogicNet-M	105,187	612	23,020,000
LILogicNet-L	51,101	302	-

* Results not reported in the original work; we reproduced the architecture using our framework with the same training setup and training time.
- Could not fit on FPGA.

4.4. Comparison with previous work

Table 6 compares LILogicNet with prior logic-based models in terms of accuracy, gate count, and FPGA throughput across MNIST, FashionMNIST, and CIFAR-10. Overall, LILogicNet achieves competitive accuracy while requiring substantially fewer gates than prior LGN architectures. For example, on MNIST, LILogicNet-M achieves 98.45% accuracy with only 8K gates, compared to 384K gates for DiffLogicNet with similar accuracy. Similar trends hold on FashionMNIST, where LILogicNet slightly improves accuracy over DiffLogicNet while using significantly smaller models. On CIFAR-10, LILogicNet consistently outperforms DiffLogicNet variants under comparable gate budgets. In addition, LILogicNet achieves higher FPGA throughput than DWN despite being evaluated on a smaller FPGA and without hardware-specific optimization, indicating strong potential for efficient deployment.

LogicTreeNet differs from the remaining approaches as it adopts a CNN-like architecture, while the rest LGN models follow a feed-forward design. As a result, its larger variants achieve substantially higher accuracy but require significantly larger gate budgets. In this work, we limited our exploration to models up to 256K gates, where LILogicNet achieves competitive or better accuracy than the smallest LogicTreeNet models with fewer gates. Scaling learnable connectomes to larger architectures remains challenging due to the combinatorial growth of possible logic configurations, and will be investigated in future work.

Table 6. Accuracy vs. gate count and FPGA throughput comparison for LGN models on MNIST, CIFAR-10, and FashionMNIST.

Model	**Acc [%]**	**Gate Count**	**FPGA t. [FPS]**
MNIST			
LogicTreeNet-S [25]	98.46	147 K	**250.0 M**
LogicTreeNet-M [25]	99.23	566 K	**200.0 M**
LogicTreeNet-L [25]	**99.35**	1.27 M	-
DiffLogicNet-S [24]	97.69	48 K	-
DiffLogicNet [24]	98.47	384 K	-
RV-LGN [38]	95.80	18 K	-
eXpLogic [39]	91.20	5 K	-
DWN(n=6) [3]	98.77	-	22.2 M
LILogicNet-S (ours)	97.96	**4 K**	**40.1 M**
LILogicNet-M (ours)	98.45	8 K	32.9 M
LILogicNet-L (ours)	**98.95**	32 K	29.7 M
FashionMNIST			
DiffLogicNet-S* [24]	89.85	48 K	-
DiffLogicNet* [24]	90.24	384 K	-
DWN(n=2) [3]	89.12	-	10.0 M
DWN(n=6) [3]	89.01	-	10.0 M
LILogicNet-S (ours)	89.95	**8 K**	**32.9 M**
LILogicNet-M (ours)	90.26	64 K	23.0 M
LILogicNet-L (ours)	**90.61**	128 K	-
CIFAR-10			
LogicTreeNet-S [25]	60.38	400 K	**111.1 M**
LogicTreeNet-G [25]	86.21	61 M	-
DiffLogicNet-S [24]	51.27	48 K	-
DiffLogicNet-M [24]	57.39	512 K	-
DiffLogicNet-L [24]	60.78	1.28 M	-
DiffLogicNet-XL [24]	62.14	5.12 M	-
RV-LGN [38]	51.30	36 K	-
DWN(n=2) [3]	57.51	-	468 K
DWN(n=6) [3]	57.42	-	468 K
LILogicNet-S (ours)	55.11	**8 K**	**31.2 M**
LILogicNet-M (ours)	57.66	64 K	-
LILogicNet-L (ours)	**60.98**	256 K	-

* Results not reported in the original work; we reproduced the architecture using our framework and the same training setup and training time.
- Value not reported or could not fit on FPGA.

5. Conclusion

We introduced **LILogicNet**, a compact logic gate network that jointly learns gate functions and their interconnections. Top-K connectivity enables the model to focus on the most informative connections, improving accuracy and efficiency compared to fixed or dense designs. Basis Projection further speeds up training by up to 4× by efficiently computing gate outputs via a fixed projection. LILogicNet maintains high inference throughput across GPU, CPU, and FPGA, with compact variants delivering hundreds of thousands to tens of millions of FPS, and its performance could be further improved by hardware-specific optimizations. Future work will explore regularization and architectural reductions (e.g., gates or adders) to scale to larger models and more complex datasets. Overall, LILogicNet provides an efficient, interpretable, and hardware-friendly framework for learnable logic networks, with potential for further improvements in performance, efficiency, and scalability.

Acknowledgments. This work was supported by the Department of Artificial Intelligence at the Wrocław University of Science and Technology.

References

[1] Atish Agarwala, Jeffrey Pennington, Yann Dauphin, and Sam Schoenholz. Temperature check: theory and practice for training models with softmax-cross-entropy losses. *arXiv preprint arXiv:2010.07344*, 2020. 4

[2] Tutu Ajayi and David Blaauw. Openroad: Toward a self-driving, open-source digital layout implementation tool chain. In *Proceedings of Government Microcircuit Applications and Critical Technology Conference*, 2019. 7

[3] Alan TL Bacellar, Zachary Susskind, Mauricio Breternitz Jr, Eugene John, Lizy K John, Priscila Lima, and Felipe MG França. Differentiable weightless neural networks. *arXiv preprint arXiv:2410.11112*, 2024. 1, 2, 7, 8

[4] Colby Banbury, Vijay Janapa Reddi, Paul Torelli, et al. Mlperf tiny benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. 1

[5] M. Champs and B. Baldi. An updated survey of efficient hardware architectures for accelerating deep convolutional neural networks. *Future Internet*, 12(7):113, 2023. 1

[6] Ian Clester. Synthesizing music with logic gate networks. In *Proceedings of the International Conference on New Interfaces for Musical Expression*, pages 618–622, 2025. 2

[7] Xue Geng, Zhe Wang, Chunyun Chen, Qing Xu, Kaixin Xu, Chao Jin, et al. From algorithm to hardware: A survey on efficient and safe deployment of deep neural networks. *arXiv preprint arXiv:2405.06038*, 2024. 1

[8] GlobalFoundries and Google. Globalfoundries gf180mcu open pdk. https://github.com/google/gf180mcu-pdk, 2022. Accessed: 2026-03-07. 7

[9] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. 1

[10] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015. 1

[11] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Binarized neural networks. *Advances in neural information processing systems*, 29, 2016. 2

[12] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Quantized neural networks: Training neural networks with low precision weights and activations. *journal of machine learning research*, 18(187):1–30, 2018. 2

[13] IHP – Leibniz Institute for High Performance Microelectronics. Ihp sg13g2 open source pdk. https://github.com/IHP-GmbH/IHP-Open-PDK, 2023. Accessed: 2026-03-07. 7

[14] Norman P. Jouppi, Cliff Young, Nishant Patil, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*, pages 1–12. ACM, 2017. 1

[15] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015. 3, 4

[16] Fabian Kresse, Emily Yu, Christoph H Lampert, and Thomas A Henzinger. Logic gate neural networks are good for verification. *arXiv preprint arXiv:2505.19932*, 2025. 2

[17] Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. Master's thesis, Department of Computer Science, University of Toronto, 2009. 3

[18] Ian Kuon, Russell Tessier, and Jonathan Rose. Fpga architecture: Survey and challenges. *Foundations and Trends in Electronic Design Automation*, 2(2):135–253, 2007. 1

[19] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 1

[20] Yann LeCun, Corinna Cortes, and Christopher J.C. Burges. MNIST handwritten digit database. http://yann.lecun.com/exdb/mnist/, 1998. Accessed: 2025-05-31. 3

[21] Zechun Liu, Zhiqiang Shen, Marios Savvides, and Kwang-Ting Cheng. Reactnet: Towards precise binary neural network with generalized activation functions. In *European conference on computer vision*, pages 143–159. Springer, 2020. 2

[22] Xiangzhong Luo, Di Liu, Hao Kong, Shuo Huai, Hui Chen, et al. Efficient deep learning infrastructures for embedded computing systems: A comprehensive survey and future envision. *arXiv preprint arXiv:2411.01431*, 2024. 1

[23] Diksha Moolchandani et al. Review of asic accelerators for deep neural network. *Microprocessors & Microsystems*, 89, 2022. 1

[24] Felix Petersen, Christian Borgelt, Hilde Kuehne, and Oliver Deussen. Deep differentiable logic gate networks. *Advances in Neural Information Processing Systems*, 35:2006–2018, 2022. 1, 2, 3, 4, 8

[25] Felix Petersen, Hilde Kuehne, Christian Borgelt, Julian Welzel, and Stefano Ermon. Convolutional differentiable logic gate networks. *Advances in Neural Information Processing Systems*, 37:121185–121203, 2024. 2, 8

[26] Haotong Qin, Ruihao Gong, Xianglong Liu, Mingzhu Shen, Ziran Wei, Fengwei Yu, and Jingkuan Song. Forward and backward information retention for accurate binary neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2250–2259, 2020. 2

[27] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. Xnor-net: Imagenet classification using binary convolutional neural networks. In *European conference on computer vision*, pages 525–542. Springer, 2016. 2

[28] Charles H Roth Jr and Larry L Kinney. *Fundamentals of Logic Design*. Cengage Learning, 2004. 3

[29] Connor Shorten and Taghi M Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of big data*, 6(1):1–48, 2019. 4

[30] Cristina Silvano, Daniele Ielmini, Fabrizio Ferrandi, Leandro Fiorin, et al. A survey on deep learning hardware accelerators for heterogeneous hpc platforms. *arXiv preprint arXiv:2306.15552*, 2023. 1

[31] Patrice Y Simard, David Steinkraus, John C Platt, et al. Best practices for convolutional neural networks applied to visual document analysis. In *Icdar*. Edinburgh, 2003. 4

[32] SkyWater Technology and Google. Skywater sky130 open pdk. https://github.com/google/skywater-pdk, 2020. Accessed: 2026-03-07. 7

[33] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S Emer. Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, 105(12):2295–2329, 2017. 1

[34] Yaman Umuroglu, Nicholas J Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. Finn: A framework for fast, scalable binarized neural network inference. In *Proceedings of the 2017 ACM/SIGDA international symposium on field-programmable gate arrays*, pages 65–74, 2017. 2

[35] Yaman Umuroglu, Yash Akhauri, Nicholas James Fraser, and Michaela Blott. Logicnets: Co-designed neural networks and circuits for extreme-throughput applications. In *2020 30th International Conference on Field-Programmable Logic and Applications (FPL)*, pages 291–297. IEEE, 2020. 2

[36] Emile Van Krieken, Erman Acar, and Frank Van Harmelen. Analyzing differentiable fuzzy logic operators. *Artificial Intelligence*, 302:103602, 2022. 3

[37] Erwei Wang, James J Davis, Peter YK Cheung, and George A Constantinides. Lutnet: Rethinking inference in fpga soft logic. In *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pages 26–34. IEEE, 2019. 2

[38] Xingbo Wang, Chenxi Feng, Xinyu Kang, Yuru Li, Yucong Huang, and Terry Tao Ye. Logic gate network inference acceleration with risc-v custom instruction set. In *Proceedings of the 22nd ACM International Conference on Computing Frontiers*, pages 205–211, 2025. 2, 8

[39] Stephen Wormald, David Koblah, Matheus Kunzler Maldaner, Domenic Forte, and Damon L Woodard. explogic: Explaining logic types and patterns in difflogic networks. In *International Conference on Information Technology-New Generations*, pages 282–292. Springer, 2025. 2, 8

[40] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017. 3

[41] Shakir Yousefi, Andreas Plesner, Till Aczel, and Roger Wattenhofer. Mind the gap: Removing the discretization gap in differentiable logic gate networks. *arXiv preprint arXiv:2506.07500*, 2025. 2

[42] Chunyu Yuan and Sos S Agaian. A comprehensive review of binary neural network. *Artificial Intelligence Review*, 56(11): 12949–13013, 2023. 2

[43] Chang Yue and Niraj K Jha. Learning interpretable differentiable logic networks. *IEEE Transactions on Circuits and Systems for Artificial Intelligence*, 2024. 2

[44] Chang Yue and Niraj K Jha. Learning interpretable differentiable logic networks for tabular regression. *arXiv preprint arXiv:2505.23615*, 2025. 2

[45] Lotfi A Zadeh. Fuzzy sets. *Information and Control*, 8(3): 338–353, 1965. 3

[46] Xia Zhao, Limin Wang, Yufei Zhang, Xuming Han, Muhammet Deveci, and Milan Parmar. A review of convolutional neural networks in computer vision. *Artificial Intelligence Review*, 57(4):99, 2024. 1

[47] Renaldas Zioma. Lgn fashionmnist implementation (source code). https://github.com/rejunity/ws0-lgn-fxnist/, 2025. Accessed: 2026-04-10. 7

[48] Renaldas Zioma. Lgn fashionmnist chip tape-out (gf180mcu gds). https://gitlab.com/rejunity/ws0-lgn-fxnist-gf180mcu-tapeout/, 2025. Accessed: 2026-04-10. 7

[49] Renaldas Zioma and Jogundas Armaitis. Lgn mnist implementation (tinytapeout project). https://github.com/rejunity/tt10-lgn-mnist, 2025. Accessed: 2026-04-10. 7

[50] Renaldas Zioma and Jogundas Armaitis. Lgn mnist chip on tinytapeout (ttsky25a). https://tinytapeout.com/chips/ttsky25a/tt_um_rejunity_lgn_mnist/, 2025. Accessed: 2026-04-10. 7

LILogic Net: Compact Logic Gate Networks with Learnable Connectivity for Efficient Hardware Deployment

Supplementary Material

6. Full Experimental Results on MNIST

Table 7 presents the complete set of results from our experiments on the MNIST dataset, expanding on the summary shown in the main paper.

We systematically varied the **layer width**, **network depth** (1 to 4 layers), and **logical connectivity strategy**, including:

- **F**: Fixed, non-learnable connectome,
- **Top-K**: Top-K sparse, learnable connectome with $K \in \{2, 4, 8, 16, 32, 64, 128\}$,
- **L**: Dense, fully learnable connectome.

For each architecture, we report:

- **Test accuracy** (mean $\pm$ std. deviation - in %),
- **Training time** (mean $\pm$ std. deviation - in minutes).

6.1. Experimental Setup

- Each configuration was run 10 times for 1- and 2-layer models, and 5 times for 3- and 4-layer networks.
- Training time was measured on a single NVIDIA A4000 GPU and includes model validation every 25 epochs (200 in total).
- Entries with "–" indicate configurations that were infeasible due to memory constraints.

6.2. Key Observations

Learned sparse Top-K connectivity consistently outperforms fixed random connectivity, especially at smaller model widths, with the performance gap narrowing as the width increases. At lower widths (e.g., 2K), the accuracy difference between Top-16 and fixed random (F) connectivity is more pronounced (over 2% points difference at 4 layers). As width increases to 32K, both methods achieve high accuracy, with Top-K maintaining a modest advantage (around 0.3–0.4% point). This indicates that while fixed random wiring can suffice at large scales, learned sparse wiring helps especially in smaller or medium-width models by optimizing logical connections more effectively.

Sparse Top-K connectivity improves accuracy across widths, with depth effects depending on K. Increasing the number of inputs K per gate generally improves accuracy across all widths (2K to 32K). For smaller K values (e.g., Top-2, Top-4), increasing the number of layers consistently enhances performance, reflecting the benefit of deeper feature integration. However, for larger K values (e.g., Top-64, Top-128), the best accuracy is often achieved at 2 or 3 layers, with 4-layer models showing little or no further improvement—or even slight degradation. This suggests that when gates have access to more inputs, very deep architectures may lead to overfitting or diminishing returns in this setting.

Top-K offers an efficient trade-off between accuracy and computational cost. Top-K connectivity sparsifies each gate's inputs, significantly reducing the number of active computations during both training and inference, while retaining high accuracy. For instance, at 8K width and 2 layers, a Top-16 model achieves **98.66%** accuracy — slightly exceeding fully learnable (L) models — while using fewer connections and benefiting from reduced computational complexity. This makes Top-K particularly attractive for efficient, scalable architectures in resource-constrained settings.

Comparison of fully learned (L) and Top-K connectivity within the same width and depth. At shallow depths (e.g., 1 layer), learned connectivity (L) consistently achieves the highest accuracy, outperforming both fixed random (F) and Top-K sparsity patterns. However, as the network depth increases, Top-K connectivity increasingly surpasses learned connectivity, demonstrating stronger scalability and better generalization in deeper models. Interestingly, for deeper architectures, smaller values of K in Top-K (e.g., Top-8, Top-16) tend to yield the best accuracy, suggesting that moderate sparse wiring is sufficient for effective information integration while controlling complexity and overfitting.

At fixed gate budgets, shallow and wide architectures often outperform deeper ones — until a certain scale is reached. As discussed in Section 4.3, under constrained gate budgets (2K–8K), shallow fully learnable (L) models achieve surprisingly strong performance. For instance, a 1-layer 4K-width model (1L–4K) reaches **97.96%** accuracy, outperforming deeper or sparse Top-K models of similar budget (e.g., 2Top128–2K with **97.66%**). This suggests that when capacity is limited, wider layers and dense training provide more effective use of resources than depth or sparsity. However, at higher budgets (e.g., 8K+), Top-K architectures begin to outperform learned ones. For example, 2Top64–8K reaches **98.69%**, exceeding 1L–16K (**98.58%**). Depth becomes beneficial beyond a threshold, but results show that 2-layer configurations often outperform their 4-layer counterparts (e.g., 2Top64–8K vs. 4Top16–4K), indicating diminishing returns from excessive depth once sparsity and width are well-balanced.

Training time scales non-uniformly with width and K, but grows roughly linearly with depth. Training time increases with model width, but the scaling is non-monotonic: from 2K to 8K gates, the time increase is modest, likely

Width	Type	1 Layer		2 Layers		3 Layers		4 Layers	
		Test Acc. [%]	Train Time [min]	Test Acc. [%]	Train Time [min]	Test Acc. [%]	Train Time [min]	Test Acc. [%]	Train Time [min]
2,000	F	87.09 ± 0.28	3.3 ± 0.3	90.82 ± 0.40	5.5 ± 0.6	93.69 ± 0.29	7.4 ± 0.4	95.38 ± 0.20	8.9 ± 0.7
	Top2	90.05 ± 0.18	4.3 ± 0.2	94.58 ± 0.21	1.9 ± 0.0	96.25 ± 0.12	2.5 ± 0.0	96.73 ± 0.12	3.1 ± 0.0
	Top4	92.41 ± 0.23	4.6 ± 0.3	96.39 ± 0.15	2.0 ± 0.0	97.19 ± 0.12	2.7 ± 0.1	97.38 ± 0.14	3.3 ± 0.1
	Top8	94.29 ± 0.12	4.4 ± 0.3	96.97 ± 0.12	2.0 ± 0.0	97.47 ± 0.16	2.7 ± 0.0	97.59 ± 0.24	3.4 ± 0.0
	Top16	95.63 ± 0.19	4.6 ± 0.3	97.35 ± 0.08	2.6 ± 0.0	97.72 ± 0.12	4.0 ± 0.0	**97.75 ± 0.12**	5.3 ± 0.0
	Top32	96.38 ± 0.15	4.6 ± 0.2	97.50 ± 0.13	4.8 ± 0.0	97.83 ± 0.15	7.8 ± 0.0	97.71 ± 0.12	10.8 ± 0.0
	Top64	96.86 ± 0.18	5.1 ± 0.3	97.65 ± 0.05	8.9 ± 0.0	**97.89 ± 0.06**	15.1 ± 0.0	97.45 ± 0.31	21.3 ± 0.0
	Top128	97.02 ± 0.12	5.6 ± 0.1	**97.66 ± 0.06**	17.6 ± 0.0	97.67 ± 0.17	23.3 ± 0.0	96.92 ± 0.14	37.5 ± 6.5
	L	**97.25 ± 0.13**	5.0 ± 0.4	97.37 ± 0.08	4.1 ± 0.0	96.13 ± 0.52	6.4 ± 0.0	92.00 ± 1.54	8.5 ± 0.1
4,000	F	90.16 ± 0.24	1.4 ± 0.6	93.28 ± 0.21	1.9 ± 0.7	95.77 ± 0.18	2.4 ± 1.0	96.90 ± 0.11	2.7 ± 0.0
	Top2	92.37 ± 0.14	1.5 ± 0.4	96.50 ± 0.15	2.0 ± 0.1	97.56 ± 0.13	2.6 ± 0.1	97.84 ± 0.15	3.2 ± 0.0
	Top4	94.66 ± 0.26	1.5 ± 0.0	97.61 ± 0.10	2.2 ± 0.1	97.99 ± 0.10	2.9 ± 0.0	98.04 ± 0.21	3.7 ± 0.0
	Top8	96.27 ± 0.11	1.5 ± 0.0	97.91 ± 0.09	2.8 ± 0.0	98.13 ± 0.07	4.2 ± 0.0	98.22 ± 0.11	5.7 ± 0.0
	Top16	97.17 ± 0.10	1.9 ± 0.0	98.14 ± 0.15	4.6 ± 0.1	98.33 ± 0.09	7.3 ± 0.0	**98.37 ± 0.09**	10.1 ± 0.0
	Top32	97.54 ± 0.16	3.1 ± 0.9	98.17 ± 0.16	8.3 ± 0.1	**98.38 ± 0.05**	13.8 ± 0.0	98.29 ± 0.12	19.4 ± 0.1
	Top64	97.76 ± 0.09	6.3 ± 0.0	98.25 ± 0.07	19.0 ± 0.0	98.31 ± 0.09	31.5 ± 0.1	98.06 ± 0.18	44.1 ± 0.0
	Top128	97.94 ± 0.07	8.0 ± 0.0	**98.28 ± 0.08**	33.1 ± 0.2	98.30 ± 0.07	58.8 ± 0.0	97.56 ± 0.16	83.6 ± 0.1
	L	**97.96 ± 0.11**	2.5 ± 0.0	98.11 ± 0.06	10.8 ± 0.0	97.02 ± 0.53	19.2 ± 0.1	94.41 ± 1.49	27.4 ± 0.3
8,000	F	91.48 ± 0.18	1.1 ± 0.0	94.83 ± 0.12	1.6 ± 0.0	96.83 ± 0.11	2.2 ± 0.0	97.69 ± 0.12	2.8 ± 0.0
	Top2	93.72 ± 0.09	1.2 ± 0.0	97.41 ± 0.07	2.7 ± 0.0	98.20 ± 0.11	4.0 ± 0.0	98.28 ± 0.14	5.3 ± 0.0
	Top4	95.70 ± 0.12	1.4 ± 0.0	98.24 ± 0.08	3.4 ± 0.0	98.50 ± 0.04	5.4 ± 0.0	98.56 ± 0.05	7.3 ± 0.0
	Top8	97.17 ± 0.07	1.8 ± 0.0	98.43 ± 0.07	5.7 ± 0.0	98.60 ± 0.06	9.4 ± 0.0	98.63 ± 0.09	13.1 ± 0.0
	Top16	97.85 ± 0.08	2.8 ± 0.0	98.66 ± 0.06	10.3 ± 0.0	98.69 ± 0.04	17.6 ± 0.1	**98.79 ± 0.07**	24.9 ± 0.1
	Top32	98.15 ± 0.02	6.3 ± 0.0	98.63 ± 0.08	22.8 ± 0.1	**98.77 ± 0.13**	39.2 ± 0.1	98.72 ± 0.05	55.5 ± 0.2
	Top64	98.36 ± 0.07	11.6 ± 0.0	**98.69 ± 0.06**	45.3 ± 0.2	98.74 ± 0.09	78.5 ± 0.3	98.44 ± 0.10	111.5 ± 0.4
	Top128	98.39 ± 0.07	16.2 ± 0.1	98.67 ± 0.06	82.4 ± 0.4	98.72 ± 0.11	148.1 ± 0.3	97.94 ± 0.28	213.8 ± 0.6
	L	**98.45 ± 0.07**	4.3 ± 0.1	98.62 ± 0.12	4.4 ± 0.1	97.87 ± 0.22	69.2 ± 0.2	96.81 ± 0.77	101.4 ± 0.2
16,000	F	93.16 ± 0.16	1.3 ± 0.0	96.43 ± 0.12	2.6 ± 0.0	97.82 ± 0.13	3.8 ± 0.0	98.34 ± 0.08	5.1 ± 0.0
	Top2	95.52 ± 0.16	1.8 ± 0.0	98.15 ± 0.09	6.0 ± 0.1	98.53 ± 0.03	10.0 ± 0.0	98.64 ± 0.09	14.0 ± 0.0
	Top4	97.24 ± 0.09	2.2 ± 0.0	98.59 ± 0.08	8.8 ± 0.0	98.68 ± 0.06	15.3 ± 0.0	98.77 ± 0.06	21.8 ± 0.1
	Top8	98.15 ± 0.04	3.1 ± 0.0	98.74 ± 0.10	14.7 ± 0.1	98.80 ± 0.08	25.9 ± 0.1	**98.87** ± 0.08	37.3 ± 0.4
	Top16	98.48 ± 0.10	6.9 ± 0.0	98.80 ± 0.05	30.2 ± 0.1	98.79 ± 0.08	53.4 ± 0.2	98.83 ± 0.10	76.7 ± 0.5
	Top32	98.50 ± 0.07	12.2 ± 0.0	**98.95 ± 0.09**	57.1 ± 0.2	**98.86 ± 0.07**	101.6 ± 0.3	98.73 ± 0.05	146.2 ± 0.5
	Top64	98.53 ± 0.11	22.5 ± 0.0	98.88 ± 0.06	111.9 ± 0.3	98.85 ± 0.10	200.7 ± 0.4	98.71 ± 0.04	289.1 ± 0.3
	Top128	98.55 ± 0.03	42.3 ± 0.0	98.89 ± 0.11	219.4 ± 0.3	98.84 ± 0.09	396.5 ± 0.3	98.23 ± 0.12	475.6 ± 0.5
	L	**98.58** ± 0.07	7.8 ± 0.0	98.82 ± 0.10	154.9 ± 0.1	–	–	–	–
32,000	F	94.74 ± 0.07	2.9 ± 0.0	97.41 ± 0.13	5.4 ± 0.0	98.27 ± 0.10	8.0 ± 0.1	98.52 ± 0.09	10.4 ± 0.2
	Top2	96.85 ± 0.06	3.1 ± 0.0	98.41 ± 0.12	15.8 ± 0.1	98.64 ± 0.11	28.7 ± 0.2	98.58 ± 0.06	41.0 ± 0.7
	Top4	98.05 ± 0.11	4.0 ± 0.0	98.67 ± 0.05	23.0 ± 0.1	98.81 ± 0.04	43.6 ± 0.3	98.79 ± 0.08	61.4 ± 0.6
	Top8	98.41 ± 0.11	7.7 ± 0.0	98.75 ± 0.07	40.5 ± 0.1	98.78 ± 0.01	69.1 ± 0.3	98.76 ± 0.06	105.5 ± 0.3
	Top16	98.56 ± 0.07	13.4 ± 0.1	98.83 ± 0.04	57.9 ± 1.0	98.83 ± 0.05	103.0 ± 0.5	98.81 ± 0.05	149.5 ± 0.5
	Top32	**98.58 ± 0.07**	24.2 ± 0.1	98.83 ± 0.09	134.3 ± 0.1	98.84 ± 0.10	240.7 ± 1.2	**98.89 ± 0.06**	348.4 ± 0.7
	Top64	98.55 ± 0.10	45.5 ± 0.1	98.87 ± 0.07	268.0 ± 0.8	**98.87 ± 0.05**	490.2 ± 6.6	97.04 ± 0.04	723.0 ± 1.2
	Top128	98.51 ± 0.08	84.0 ± 0.1	**98.91 ± 0.05**	524.1 ± 0.2	98.83 ± 0.04	884.5 ± 4.2	97.77 ± 0.20	1326.6 ± 5.1
	L	98.52 ± 0.09	15.0 ± 0.1	–	–	–	–	–	–

Table 7. Test accuracy and training time on MNIST for different logic gate networks, layer widths, and depths. Each cell shows the mean ± standard deviation over multiple runs (10 runs for 1-2 layers, 5 runs for 3-4 layers). Dashes indicate inapplicable configurations. Bolded values indicate the best performance per block.

due to better GPU utilization. However, for larger widths (16K–32K), the increase becomes steeper, suggesting that memory or computational bottlenecks begin to dominate. The relationship between training time and the number of gate inputs K is not strictly linear: while larger K generally increases cost, the overhead is influenced by implementation details, such as sparse gather operations and input aggregation. Top-128 models are much slower than Top-16, but the scaling trend is irregular. In contrast, training time grows approximately linearly with depth — adding layers increases computation in a predictable way. Notably, fully learnable connectivity (L) often trains faster than Top-K models despite involving weight optimization. This is likely due to GPU-accelerated dense matrix operations (`matmul`) being more efficient than sparse input selection used in Top-K. Several anomalies are observed: (i) For width 2K, 1-layer model, training times are longer than for 4K or 8K models. This may stem from poor GPU utilization, kernel launch overhead, or small matrix sizes being inefficiently handled. (ii) Fixed connectivity (F) also shows unexpectedly long training times at low widths (e.g., 2K), possibly due to inefficient memory access or unoptimized computation of fixed connections.

Low variance in results confirms robustness. Across all settings, standard deviation in test accuracy remains under 0.2%, indicating stable convergence and reproducibility, even for models with high structural sparsity.

7. Full Experimental Results on CIFAR-10

Table 8 presents the complete set of results from our experiments on the CIFAR-10 dataset, expanding on the summary shown in the main paper.

Building on insights from MNIST, we focused on 1-layer variants for smaller architectures (8K gates) and 1–2-layer variants for wider layers. We evaluated the following logical connectivity strategies:

- **F**: Fixed, non-learnable connectome,
- **Top-K**: Top-K sparse, learnable connectome with $K = 32$, chosen based on MNIST experiments as a good trade-off between accuracy and training time,
- **L**: Dense, fully learnable connectome.

We also experimented with two sets of fixed thresholds to binarize each RGB channel:

- $[0.125, 0.25, 0.375, 0.5, 0.625, 0.75, 0.875]$,
- $[0.2, 0.4, 0.6, 0.8]$,

resulting in input vector sizes of 21,504 and 12,288, respectively.

For each architecture, we report:

- τ **value**,
- **Test accuracy** (mean ± std. deviation in
- **Training time** (mean ± std. deviation in minutes).

7.1. Experimental Setup

- Each configuration was run 5 times, and results are averaged.
- Training time was measured on a single NVIDIA H200 GPU and includes model validation every 25 epochs (200 in total).
- The entire training dataset was loaded at the beginning instead of subsequent batch loading.
- Entries with "–" indicate configurations that were infeasible due to memory constraints.

7.2. Key Observations

Most of the observations reported for MNIST also apply to CIFAR-10. Therefore, here we focus only on additional aspects specific to this dataset.

Threshold count has limited effect. The difference between using 4 or 7 thresholds is small, though the 7-threshold configuration shows a slight advantage in most cases.

Training time is largest for L-type architectures. For MNIST, L-type layers often trained faster than Top-K models, likely because GPU-accelerated dense matrix operations (`matmul`) are more efficient than sparse input selection used in Top-K. In CIFAR-10, however, L-type layers have the longest training times due to the much larger input size, which increases the number of connections in the first layer. Additionally, training time differences between the 4- and 7-threshold modes appear only for L-type architectures for the same reason; Top-K and F architectures are unaffected, as their number of connections does not depend on input size.

Accuracy standard deviation is larger than for MNIST. Accuracy across 5 runs varies more than for MNIST, with standard deviations reaching up to 0.3%. This is largely due to binarizing separate RGB channels with multiple thresholds, which increases variability in the input representation. Although CIFAR-10 has the same number of classes and slightly larger images than MNIST, this multi-threshold binarization amplifies sensitivity to initialization, data order, and training dynamics. Noise, labeling inconsistencies, and the small number of runs also contribute, whereas MNIST's simple grayscale images with a single threshold yield highly stable results.

Best-performing architectures. Across all settings, 1-layer L-type variants achieved the highest accuracy. For the largest budget architectures (e.g., 256K gates), memory constraints prevented evaluating L-type networks; in these cases, the best results were obtained using 2-layer Top-32 networks.

#gates	Model	Layer Width	τ	7 thresholds		4 thresholds	
				Test Acc. [%]	Train Time [min]	Test Acc. [%]	Train Time [min]
8,000	1F	8,000	20	44.08 ± 0.12	2.5 ± 0.0	44.18 ± 0.27	1.3 ± 0.0
	1Top32	8,000	20	52.27 ± 0.20	2.6 ± 0.0	51.86 ± 0.26	2.5 ± 0.0
	1L	8,000	20	**55.11** ± 0.17	45.4 ± 0.1	54.51 ± 0.25	26.1 ± 0.0
64,000	1F	64,000	90	49.17 ± 0.14	2.9 ± 0.0	49.06 ± 0.25	2.0 ± 0.1
	1Top32	64,000	90	57.28 ± 0.30	15.3 ± 0.0	56.81 ± 0.19	16.9 ± 0.3
	1L	64,000	90	**57.66** ± 0.17	139.7 ± 0.1	57.64 ± 0.23	80.9 ± 0.1
	2Top32	32,000	70	57.12 ± 0.11	72.9 ± 0.0	56.40 ± 0.17	72.7 ± 0.1
	2L	32,000	70	55.75 ± 0.18	224.1 ± 0.1	55.58 ± 0.42	188.3 ± 0.4
128,000	1F	128,000	100	50.92 ± 0.24	3.8 ± 0.0	50.87 ± 0.30	3.7 ± 0.1
	1Top32	128,000	100	59.43 ± 0.22	29.3 ± 0.0	58.95 ± 0.14	30.7 ± 0.1
	1L	128,000	100	–	–	60.13 ± 0.14	157.3 ± 0.3
	2Top32	64,000	90	58.88 ± 0.26	146.7 ± 0.0	58.36 ± 0.22	146.5 ± 0.0
256,000	1F	256,000	110	52.48 ± 0.28	5.0 ± 0.0	52.05 ± 0.09	5.3 ± 0.0
	2F	128,000	100	54.76 ± 0.27	16.3 ± 0.0	54.18 ± 0.24	16.1 ± 0.0
	2Top32	128,000	100	**60.98** ± 0.19	297.2 ± 6.6	59.86 ± 0.23	298.7 ± 0.3

Table 8. Test accuracy and training time on CIFAR-10 for different logic gate networks, layer widths, depths and input binarization thresholds. Each cell shows the mean ± standard deviation over 5 runs. Dashes indicate inapplicable configurations. Bolded values indicate the chosen models (LILogic Net-S, -M, -L) with best performance.

8. Training Accuracy and Effect of Discretization

We analyze the influence of discretization on network performance by measuring the difference between the accuracy in inference mode and the accuracy during differentiable training. Early in training, this binarization effect can noticeably affect accuracy as the network starts by learning a probabilistic, differentiable LGN, with high uncertainty in the selection of individual logic gates. The discretization step—selecting the most probable gate for inference—can therefore produce substantial changes in network behavior during this phase.

As training progresses, the gate selection probabilities become more confident, and the effect of discretization on accuracy diminishes (e.g. Fig. 7). By the end of training, the difference between inference-mode and training-mode accuracy is negligible, with a final binarization error **below 0.1 percentage points**.

This pattern demonstrates that the network first explores a smooth, probabilistic logic representation and gradually converges to stable discrete logic choices, with minimal impact on final performance.

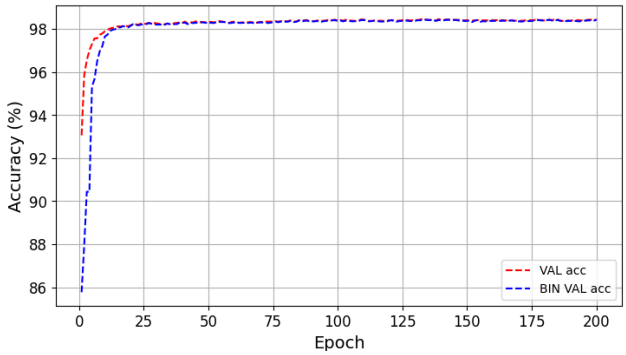


Figure 7. MNIST LILogic Net-S training and validation accuracy curves before (VAL acc) and after (BIN VAL acc) discretization. The discretization error is very small during late training.

9. LGN Model FPGA Resource Usage

Table 9 summarizes the accuracy and FPGA resource utilization of different LGN configurations across MNIST, FashionMNIST, and CIFAR-10 datasets, including LUT usage for generic and target devices.

Tables 9 and 10 summarize the hardware characteristics of various LGN configurations across MNIST, FashionMNIST, and CIFAR-10 datasets. Table 9 reports accuracy and FPGA resource utilization (LUT usage for generic and Xilinx devices), while Table 10 reports Xilinx frequency and logic/routing latency. Architectures marked with $*$ correspond to DiffLogicNet variants. Dashes (-) indicate missing or not applicable data.

Table 9. Accuracy of single runs and FPGA resources per dataset. Most of the achitectures are binarized LILogicNet-S,-M,-L. $*$ Architectures of DiffLogicNet-S and DiffLogiCNet reported for MNIST. Given two values of LUTs means total LUTS and LUTs without population count (popcount) part.

Dataset	Architecture	Acc [%]	Generic LUT4	ICE40 LUT4	Generic LUT6	Xilinx LUT6
MNIST	1x4000	97.63	11987 / 3637	11188	8760 / 3637	7103 / 1791
	1x8000	98.47	21677 / 7047	20696	16811 / 7047	14076 / 3491
	2x16000	98.94	47869 / 14940	46379	41258 / 14940	37373 / 11833
	4x8000	98.79	24676 / 8681	23697	20806 / 8065	16004 / 4637
FMNIST	1x8000	90.03	32436 / 7695	27163	-	14321 / 3736
	2x16000	90.03	51311 / 14891	50893	-	37075 / 11535
	2x32000	90.39	95342 / 29111	93926	-	73938 / 22308
	2x64000	90.66	176882 / 55302	172196	-	143226 / 39769
	6x8000*	88.56	39347 / 17562	39417	-	22710 / 10377
	6x64000*	90.27	264873 / 131992	264982	-	172787 / 72807
CIFAR10	1x8000	54.89	43237 / 7834	-	38225 / 7834	14415 / 3830
	1x64000	58.06	186010 / 57329	187468	-	104853 / 27070
	2x128000	60.76	387156 / 117164	-	339376 / 117164	293285 / 92353

Table 10. Xilinx XC7A200T frequency and latency per dataset and model architecture. Most of the achitectures are binarized LILogicNet-S,-M,-L. $*$ Architectures of DiffLogicNet-S and DiffLogicNet reported for MNIST. Dashes (-) indicate missing or not applicable data.

Dataset	LGN	Xilinx Freq	Logic Latency (ns)	Routing Latency (ns)
MNIST	1x4000	40.15 MHz	5.1	19.8
	1x8000	32.89 MHz	6.0	24.4
	2x16000	29.71 MHz	6.2	27.4
	4x8000	26.69 MHz	5.0	32.4
FMNIST	1x8000	33.76 MHz	5.9	23.8
	2x16000	20.13 MHz	6.1	43.6
	2x32000	18.16 MHz	-	-
	2x64000	-	-	-
	6x8000*	19.11 MHz	-	-
	6x64000*	-	-	-
CIFAR10	1x8000	31.18 MHz	5.8	26.2
	1x64000	-	-	-
	2x128000	-	-	-